

Lab_esp_hannah_nakamura

Hannah miranda Nakamura (249491)

Laboratório Especial

Benilton Carvalho

16-20 de Setembro de 2019

Atividade

Ao longo desta semana, você deverá desenvolver código em R, utilizando-se de tidyverse, ggplot, leaflet, ggcal e outros pacotes disponibilizados em HTML Widgets para responder uma questão relacionada ao conjunto de dados flights. Para tanto, considere os seguintes fatos:

- Cias Aéreas (AIRLINE) possuem diferentes aeronaves (codificadas em TAIL_NUMBER);
- (Exemplo hipotético) No dia 01/01/2015, a aeronave N431WN viaja de BWI para JFK;
- (Exemplo hipotético) No dia 02/01/2015, a aeronave N431WN viaja de JFK para ORD;
- E assim sucessivamente. . . No caso apontado acima, pode-se ver que a aeronave N431WN realizou um trajeto (provavelmente único) durante todo o ano de 2015. Nos dias exemplificados, o trajeto foi BWI-JFK-ORD.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2     4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr       1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(ggplot2)
library(leaflet)
```

Warning: pacote 'leaflet' foi compilado no R versão 4.6.1

```
library(ggcal)
library(readr)
library(scales)
```

Anexando pacote: 'scales'

O seguinte objeto é mascarado por 'package:purrr':

discard

O seguinte objeto é mascarado por 'package:readr':

col_factor

```
library(webshot2)
```

Warning: pacote 'webshot2' foi compilado no R versão 4.6.1

Parte 1:

Crie uma função que: a. Receba um valor de TAIL_NUMBER (por exemplo, N431WN); b. Produza uma tabela (tidy) com todos os trajetos realizados pela aeronave (ordenadas por data e hora, contendo todas as colunas do arquivo flights.csv.zip); c. Produza um mapa que apresente todo o trajeto voado pela aeronave ao longo de todo o ano; o trajeto deve ser apresentado de maneira linear no tempo (i.e., segue a sequência do tempo, como no exemplo hipotético dado acima); d. O mapa deve ser decorado com estatísticas do seu interesse (por exemplo, velocidade média do voo como espessura da linha que conecta os aeroportos envolvidos no trajeto)

Parte 2:

- a. Seu código deve ser bem comentado;
- b. Os nomes das variáveis devem ser esclarecedores (por exemplo, x é um nome ruim);
- c. Seu código será avaliado por 5 de seus colegas;
- d. A parte de avaliação (descrita em c.) acontecerá na semana de 23 a 27 de setembro (com entrega obrigatória no dia 27);

Observações:

1. A função terá o argumento `tail_number` e `arquivo`; 2. O argumento `tail_number` será o código identificador do avião de interesse (por ex., N431WN); 3. O argumento `arquivo` será o nome do arquivo com caminho completo (por ex., `file.path("dados", "flights.csv.zip")`); 4. A função se chamará `analisa_aeronave`; 5. O arquivo deve ser lido por partes; 6. O mapa deve utilizar as coordenadas exatas dos aeroportos envolvidos (presentes no arquivo `airports`); 7. A função deve retornar uma lista (nomeada) com dois valores: a tabela utilizada para o gráfico e o gráfico propriamente dito; 8. O Jupyter não será suficiente para a realização da tarefa; utilize o RStudio; o laboratório continuará disponível nos horários da aula; 9. O código deverá ser submetido via Moodle até o dia 23 de setembro, às 10 da manhã (em área exclusiva a ser disponibilizada oportunamente).

Código

```
arquivo <- file.path("datasets", "flights.csv.zip") # caminho para o banco
# de dados flights (Observações 3)
# a função `file.path` cria um caminho para um arquivo que seja compatível
# com qualquer sistema operacional evitando erros de digitação manual
```

```
# função analisa aeronaves (Observações 4)
analisa_aeronave <- function(tail_number, arquivo) { # argumentos da função
                                                    # (Observações 1, 2 e 3)

  # função para processar cada chunk
  processa_chunk <- function(chunk, pos) {
    chunk |>
      filter(TAIL_NUMBER == tail_number) |>      # recebendo um valor de
                                                    # TAIL_NUMBER (Parte 1a)
    drop_na(DISTANCE, AIR_TIME)
  }
}
```

```

# lendo os dados
flights <- read_csv_chunked(
  arquivo,
  callback = DataFrameCallback$new(processa_chunk),
  chunk_size = 1e6 # arquivo lido por partes (Observações 5)
  # omitindo o argumento `col_types` o readr tenta adivinhar os tipos das
  # colunas inspecionando as primeiras linhas do arquivo
)

# produzindo uma tabela tidy (Parte 1b)
tabela <- flights |>
  arrange(YEAR, MONTH, DAY, DEPARTURE_TIME) # ordenar por data e hora

airports <- read_csv(file.path(dirname(arquivo), "airports.csv")) # dados
# contendo as coordenadas exatas dos aeroportos envolvidos (Observações 6)
airports <- airports |> drop_na(LATITUDE, LONGITUDE)

flights <- flights |>
  # juntando os dois banco de dados para conseguir fazer o mapa
  # com as coordenadas exatas (Observações 6)
  left_join(airports, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) |>
  rename(LAT_ORIGIN = LATITUDE,
    LON_ORIGIN = LONGITUDE,
    CITY_ORIGIN = CITY,
    COUNTRY_ORIGIN = COUNTRY,
    STATE_ORIGIN = STATE,
    AIRPORT_ORIGIN = AIRPORT
  ) |>
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) |>
  rename(LAT_DESTINATION = LATITUDE,
    LON_DESTINATION = LONGITUDE,
    CITY_DESTINATION = CITY,
    COUNTRY_DESTINATION = COUNTRY,
    STATE_DESTINATION = STATE,
    AIRPORT_DESTINATION = AIRPORT
  ) |>
  mutate(
    DATA = make_date(YEAR, MONTH, DAY),
    VELOCIDADE = DISTANCE/ (AIR_TIME/ 60), # estatística de velocidade
    # (Parte 1d)
    espessura = rescale(VELOCIDADE, to = c(0.5, 2)) # para conseguir variar
    # a espessura da linha de

```

```

# acordo com a velocidade
) |>
  arrange(DATA, DEPARTURE_TIME) # ordenar por data e hora

# palheta de cor para diferenciar cada voo no mapa
cores_voos <- colorFactor(
  palette = colorRampPalette(c("#1f77b4", "#ff7f0e", "#2ca02c", "#d62728",
                                "#9467bd"))(nrow(flights)),
  domain = flights$FLIGHT_NUMBER
)

# agrupamento para calcular total de pousos por localidade
pousos <- flights |>
  group_by(DESTINATION_AIRPORT, LAT_DESTINATION, LON_DESTINATION) |>
  summarise(total_pousos = n(),
            delay_mean = mean(DEPARTURE_DELAY, na.rm = TRUE),
            .groups = "drop") |>
  mutate(raio = rescale(total_pousos, to = c(5, 20)))

# criando um gradiente de cor para a quantidade de atrasos por localidade
palheta <- colorNumeric(
  palette = "YlOrRd",
  domain = pousos$delay_mean
)

# mapa com o trajeto
mapa <- leaflet(flights) |>
  addTiles()

for (linha in seq_len(nrow(flights))) {
  voo <- flights[linha, ]

  mapa <- mapa |>
    addPolylines( # linhas conectam coordenadas ao longo do ano (Parte 1c)
      lng = c(voo$LON_ORIGIN, voo$LON_DESTINATION),
      lat = c(voo$LAT_ORIGIN, voo$LAT_DESTINATION),
      color = cores_voos(voo$FLIGHT_NUMBER), # aplica cada cor de
                                                # forma individual
      weight = voo$espessura,
      opacity = 0.5,
      popup = paste0(
        "<b>Origem: ", voo$COUNTRY_ORIGIN, "-", voo$STATE_ORIGIN,

```

```

    "<br>", voo$CITY_ORIGIN, "<br>",
    "<b>Aeroporto (origem):", voo$AIRPORT_ORIGIN, "<br>",
    "<b>Destino:", voo$COUNTRY_DESTINATION, "-", voo$STATE_DESTINATION,
    "<br>", voo$CITY_DESTINATION, "<br>",
    "<b>Aeroporto (destino):", voo$AIRPORT_DESTINATION, "<br>",
    "<b>Data:", voo$DATA,
    "<b>Velocidade Média: </b>", round(voo$VELOCIDADE, 1),
    "mph (milhas por hora)"
  )
)
}

mapa <- mapa |>
  addCircleMarkers(
    data = pousos,
    lng = ~LON_DESTINATION,
    lat = ~LAT_DESTINATION,
    radius = ~raio,
    color = ~palheta(delay_mean),
    fillColor = ~palheta(delay_mean),
    fillOpacity = 0.3,
    weight = 2
  )

output <- list(tabela = tabela, grafico = mapa)

return(output) # função retorna uma lista com os valores da tabela e mapa
}

```

```

#testando a função com N431WN
teste <- analisa_aeronave("N431WN", arquivo)

```

```

-- Column specification -----
cols(
  .default = col_double(),
  AIRLINE = col_character(),
  TAIL_NUMBER = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_character(),
  DEPARTURE_TIME = col_character(),

```

```

WHEELS_OFF = col_character(),
WHEELS_ON = col_character(),
SCHEDULED_ARRIVAL = col_character(),
ARRIVAL_TIME = col_character(),
CANCELLATION_REASON = col_character()
)
i Use `spec()` for the full column specifications.

```

Warning in validateCoords(lng, lat, funcName): Data contains 49 rows with either missing or invalid lat/lon values and will be ignored

```
teste$tabela
```

```

# A tibble: 1,921 x 31
  YEAR MONTH DAY DAY_OF_WEEK AIRLINE FLIGHT_NUMBER TAIL_NUMBER
  <dbl> <dbl> <dbl>      <dbl> <chr>      <dbl> <chr>
1  2015     1     1          4 WN          1762 N431WN
2  2015     1     1          4 WN          3564 N431WN
3  2015     1     1          4 WN          3564 N431WN
4  2015     1     1          4 WN          3564 N431WN
5  2015     1     2          5 WN          1793 N431WN
6  2015     1     2          5 WN          1793 N431WN
7  2015     1     2          5 WN          1793 N431WN
8  2015     1     2          5 WN          1793 N431WN
9  2015     1     2          5 WN          1793 N431WN
10 2015     1     2          5 WN          3144 N431WN
# i 1,911 more rows
# i 24 more variables: ORIGIN_AIRPORT <chr>, DESTINATION_AIRPORT <chr>,
#   SCHEDULED_DEPARTURE <chr>, DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>,
#   TAXI_OUT <dbl>, WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>,
#   AIR_TIME <dbl>, DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>,
#   SCHEDULED_ARRIVAL <chr>, ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>,
#   DIVERTED <dbl>, CANCELLED <dbl>, CANCELLATION_REASON <chr>, ...

```

```
teste$grafico
```

